

---

# REDIS

<https://redis.io>



# REDIS

---

## OUTLINE

1. What is Redis?
2. In-memory database
3. Redis persistence
4. What is Redis used for?

# What is Redis?

---

- *Redis: RE(Remote ) DI(Dictionary ) S(Server)*
- Redis is an open source in-memory data store that can be used as a database, cache, or message broker.
- It's often used for caching web pages and reducing the load on servers.
- It offers excellent performance, with the ability to quickly read and write data to memory. Additionally, Redis supports atomic operations, making it ideal for caching scenarios where you need fast access time.

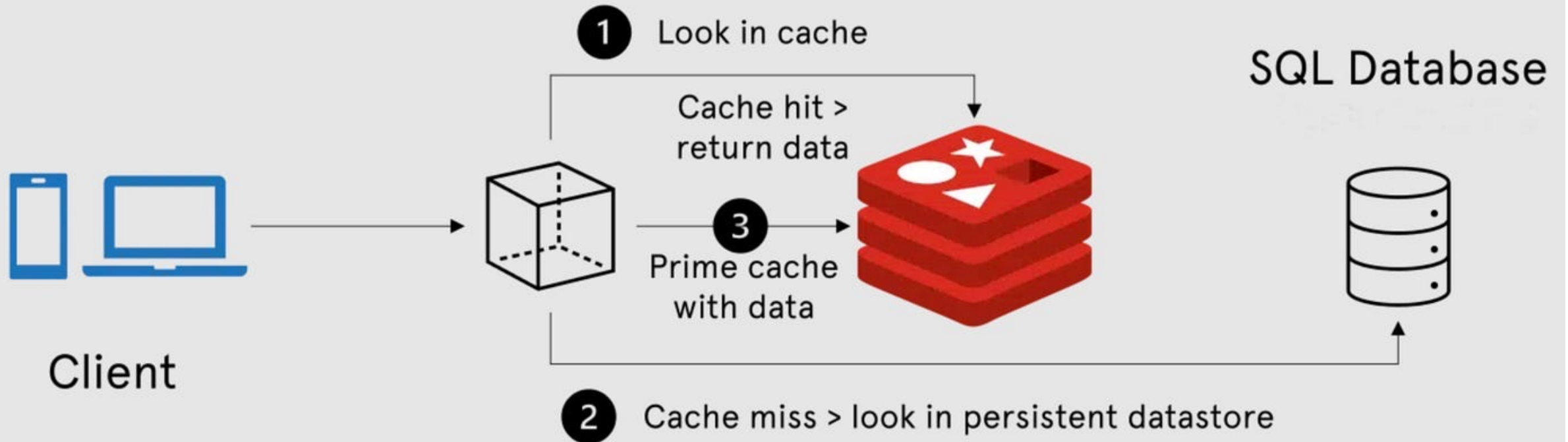
# In-memory database

---

- An in-memory database is a type of database that stores data entirely in main memory (RAM) rather than on disk. In-memory databases are designed to provide fast access to data by leveraging the high speed of main memory, which is several orders of magnitude faster than disk storage.
- In-memory databases are commonly used in applications that require fast access to large amounts of data, such as real-time analytics, online gaming, e-commerce, and social media. They are also used in applications that require high performance and scalability, as in-memory databases can handle high volumes of data and transactions without sacrificing performance.

# In-memory database (cont.)

## How Redis is typically used



## In-memory database (cont.)

---

- One of the main drawbacks of in-memory databases is that they are more sensitive to data loss in the event of a crash or shutdown, as the data is stored entirely in memory and is not persisted to disk. To address this issue, many in-memory databases, including Redis, provide features such as persistence and replication, which allow data to be saved to disk and replicated across multiple servers to ensure data durability and availability.

# Redis persistence

---

- Redis persistence is a feature of the Redis database that allows data to be saved to disk and restored in the event of a crash or shutdown. By default, Redis stores data in memory, which means that it is lost when the Redis server is shut down or restarted. Redis persistence enables data to be saved to disk and restored when the Redis server starts up again, ensuring that data is not lost in the event of a crash or shutdown.

## Redis persistence (cont.)

---

- Redis persistence can be configured in several ways, depending on the needs of the application:
  - Snapshotting: the simplest form of persistence, which involves periodically saving the entire Redis dataset to disk. This approach is fast and efficient, but it can result in data loss if the Redis server crashes between snapshots.
  - Append-only file (AOF) persistence, which involves saving each write operation to a log file on disk. This approach provides better durability than snapshotting, as it allows the Redis server to recreate the dataset by replaying the log file in the event of a crash. However, it can be slower and more resource-intensive than snapshotting.

# What is Redis used for?

---

1. **Real-time analytics:** use Redis to store and process large amounts of data in real-time, allowing organizations to quickly analyze and visualize data to make business decisions.
2. **Online gaming:** use Redis to store and manage game state, such as player profiles, game scores, and leaderboards, which allows for fast and seamless (liền mạch) gameplay.
3. **E-commerce:** use Redis to store and manage data related to online shopping, such as product catalogs, user profiles, and shopping cart contents, which enables fast and efficient shopping experiences for users.
4. **Social media:** use Redis to store and manage data related to social media interactions, such as user profiles, friend lists, and news feeds, which allows for fast and smooth user experiences.

## What is Redis used for? (cont.)

---

5. **Location-based applications:** Redis simplifies the development of location-based applications and services by providing geospatial indexing, sets, and operations. Using sorted sets, Redis is able to offload time-consuming searching and sorting of location data while also using an intelligent geo-hashing implementation.
6. **Caching for databases:** Redis is able to handle large amounts of real-time data, making use of its in-memory data storage capabilities to help support highly responsive database constructs. Caching with Redis allows for fewer database accesses, which helps to reduce the amount of traffic and instances required. By using Redis for caching, development teams can dramatically improve their application throughputs by achieving sub-millisecond latency. And since Redis' caching layer can scale quickly and economically, organizations are able to develop these highly responsive applications while reducing their overall expenditures.

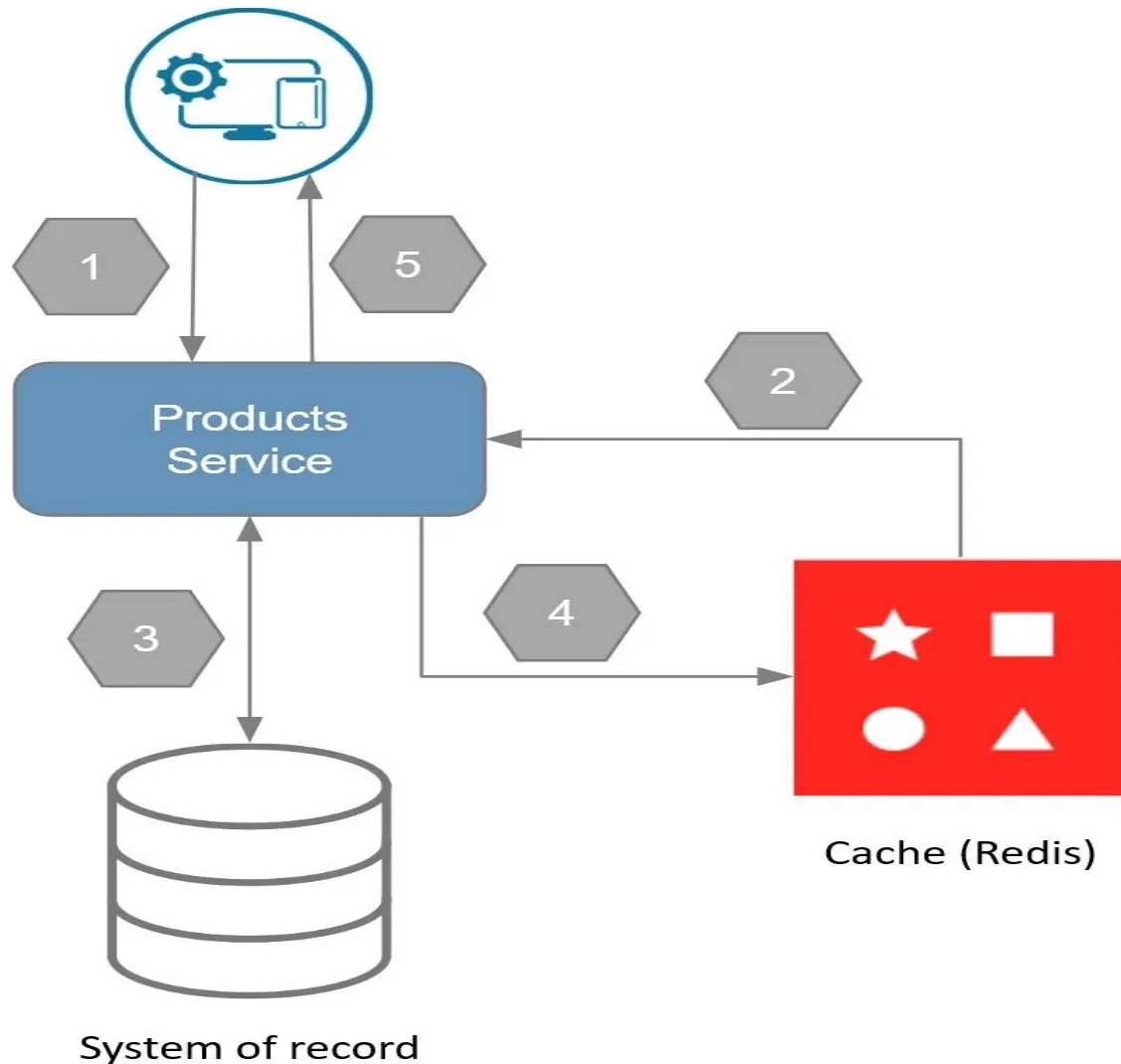
## What is Redis used for? (cont.)

---

7. **Chat and messaging applications:** To support chat and messaging applications, Redis can be used to store and manage data related to conversations, users, and messages
8. **Session store:** Session store is a mechanism for storing user session data in a web application. In a Redis session store, session data is stored in a Redis database, which is a fast, in-memory data structure store that can be used as a cache, database, and message broker.

# Example – Query caching

Search products

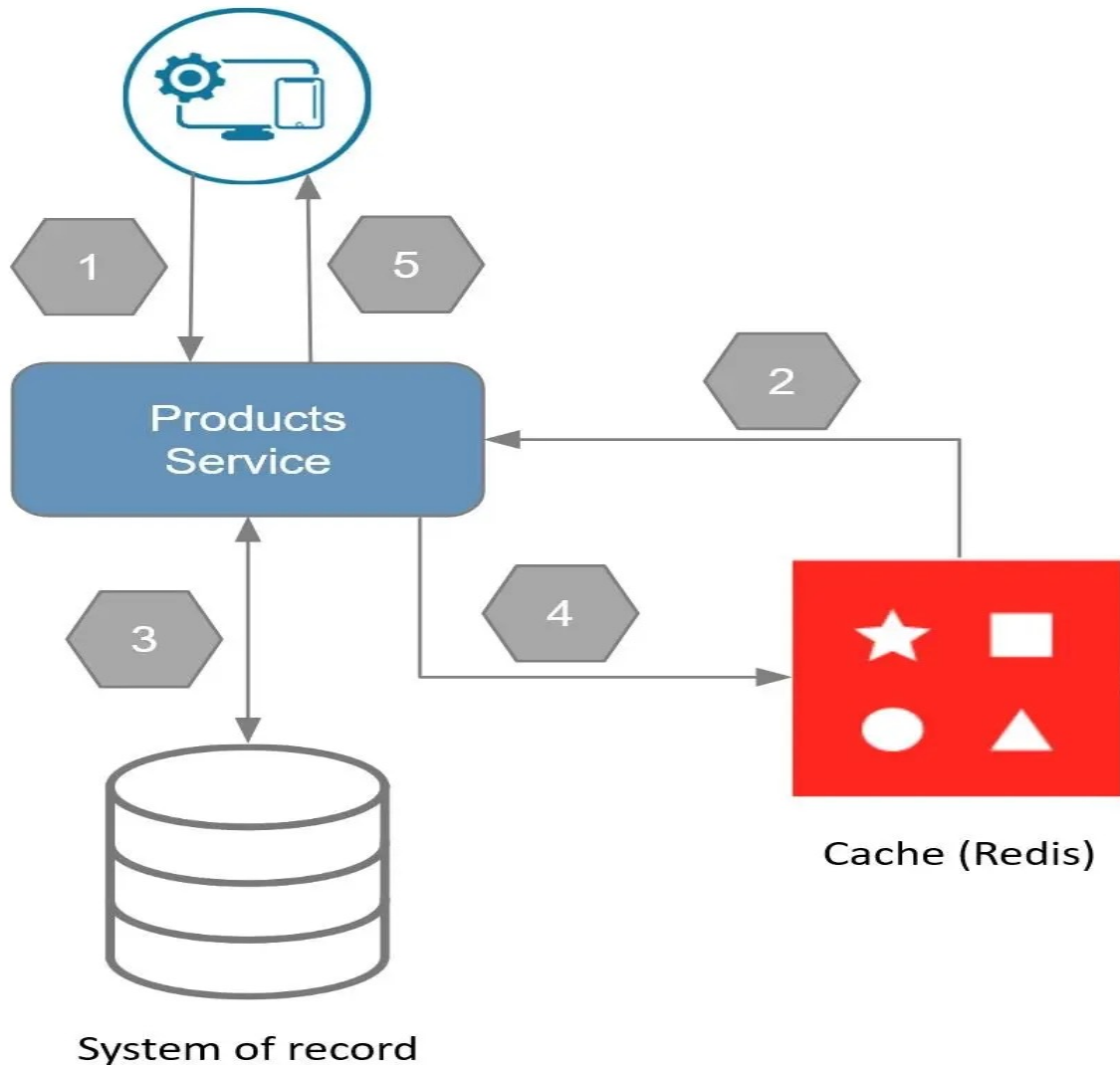


## *Cache-aside with Redis (cache miss):*

1. An application requests data from the backend.
2. The backend checks to find out if the data is available in Redis.
3. Data is not found (a cache miss), so the data is fetched from the database.
4. The data returned from the database is subsequently stored in Redis.
5. The data is then returned to the application.

# Example – Query caching (cont.)

Search products

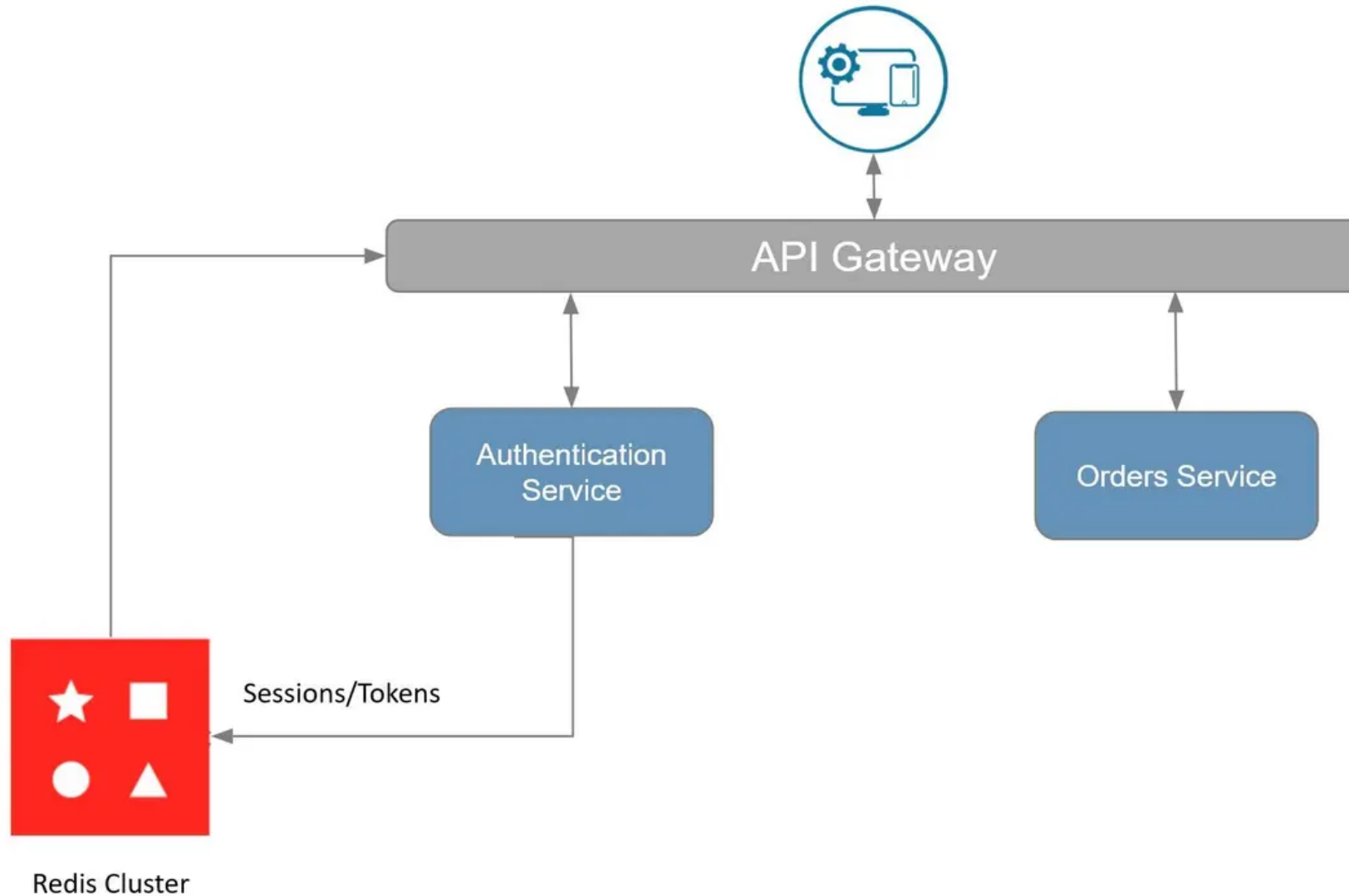


***Cache-aside with Redis (cache hit):***

1. An application requests data from the backend.
2. The backend checks to find out if the data is available in Redis.
3. The data is then returned to the application.

# Example – API Gateway caching

---



# Dependency – Spring FW

---

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-data-redis</artifactId>  
</dependency>
```

```
<dependency>  
  <groupId>org.springframework.data</groupId>  
  <artifactId>spring-data-redis</artifactId>  
</dependency>  
  
<dependency>  
  <groupId>io.lettuce.core</groupId>  
  <artifactId>lettuce-core</artifactId>  
  
  <version>6.3.2</version>  
</dependency>
```

Ref: <https://docs.spring.io/spring-data/redis/reference/redis.html>

# Dependency – Spring FW

---

**Ref:** <https://docs.spring.io/spring-framework/reference/integration/cache/annotations.html>

For caching declaration, Spring's caching abstraction provides a set of Java annotations:

- **@Cacheable**: Triggers cache population.
- **@CacheEvict**: Triggers cache eviction.
- **@CachePut**: Updates the cache without interfering with the method execution.
- **@Caching**: Regroups multiple cache operations to be applied on a method.
- **@CacheConfig**: Shares some common cache-related settings at class-level.

# Redis - Nodejs

---

*Ref: <https://redis.io/learn/howtos/caching>*

---

# Q&A